CSC 301: Algorithms

PRACTICE Midterm Exam 2 – Version D

Name: _____

Student ID: _____

Part	Marks	Score
A – Q1	20	
B – Q2	20	
B – Q3	20	
C – Q4	20	
C – Q5	20	
Total	**100**	

Instructions: Part A: circle T or F and justify in one or two lines (no marks without a justification). Part B: fill in the tables and show every step. Part C: explain your idea, write pseudocode, and justify the complexity.

PART A – CONCEPTS

QUESTION 1 – Hashing: True or False? (20 marks, 4 each)

#	Statement	T / F	Justification
1	With open addressing, the load factor $\alpha = n/m$ can be greater than 1.	T F	
2	In linear probing, deleting a key by resetting its slot to EMPTY is always safe.	T F	
3	A perfect hash table guarantees O(1) worst-case search, even for new keys inserted after the table has been built.	T F	
4	If $h(k) = k \bmod m$ with $m = 2^p$, the hash value depends only on the lowest p bits of k.	T F	
5	With chaining and $\alpha = 3$, an unsuccessful search examines about 3 elements on average.	T F	

PART B – TRACING

QUESTION 2 – Probing & Lookups (20 marks)

a) Insert **42, 12, 33, 52, 23, 7, 17** (in this order) into a table of size **m = 10** using **linear probing** with h(k) = k mod 10. **(10 marks)**

Slot	0	1	2	3	4	5	6	7	8	9
Key										

Key	42	12	33	52	23	7	17
# probes							

I. How many probes does an **unsuccessful** search for **62** take?

II. The table is only 70% full, yet some operations are slow. Name the phenomenon, explain why it happens, and name a probing method that reduces it.

b) A website logs every login as a user ID, in time order. Return the **first user who logged in exactly once**. **(10 marks)**

INPUT: `Logins = [u7, u3, u7, u9, u3, u5]` **OUTPUT:** `u9`

Explain the idea behind an efficient linear solution. Write the pseudocode and analyze its complexity.

`FirstSingleLogin(Logins):`

QUESTION 3 – Inside Counting Sort & Radix (20 marks)

a) Run counting sort on **A = [4, 1, 3, 4, 0, 1, 4]** with k = 4. **(10 marks)**

Index	0	1	2	3	4
C after counting					
C after prefix sums					

B position	1	2	3	4	5	6	7
After 3 placements							
Final B							

A student changes the last loop from `for i = n downto 1` to `for i = 1 to n`. Is the output still sorted? Is the sort still stable? Why does this matter for radix sort?

b) We must sort **n = 1,000,000** unsigned **32-bit integers** with radix sort. Each pass uses a stable counting sort on one "digit" of r bits (so 2^r buckets). **(10 marks)**

Digit choice	Buckets	# passes	Cost per pass	Total cost
Decimal digit (base 10)				
1 byte (r = 8)				
2 bytes (r = 16)				

Which choice is best, and how does it compare to an O(n log n) comparison sort? Give the general formula for the cost with b-bit keys and r-bit digits. Why not simply use r = 32?

PART C – DESIGN

QUESTION 4 – Closest Match **(20 marks)**

A music app stores song tempos (beats per minute) in a balanced BST. A DJ is playing a song at tempo **K** and asks: **"Which stored tempo is closest to K?"** (K may or may not be in the tree.)

Design an efficient algorithm to do this and give its Pseudocode & Complexity. Explain **why** the answer is guaranteed to lie on a single root-to-leaf path.

`Find-Closest(root, K):`

QUESTION 5 – Rebuild the Tree **(20 marks)**

A BST was saved to disk as its **preorder** traversal: **`[40, 25, 10, 32, 30, 60, 50, 70]`** (all keys distinct).

a) Draw the BST. **(4 marks)** **b)** Give its inorder and postorder traversals. **(4 marks)**

c) Rebuilding by inserting the keys one at a time costs $O(n^2)$ in the worst case. Design an algorithm that rebuilds the BST from its preorder in **O(n)**. Give its Pseudocode & Complexity. **(12 marks)**

`Build-From-Preorder(pre, n):`

ANSWER KEY

CSC 301 – Practice Midterm Exam 2 – Version D

PART A – QUESTION 1

#	Answer	Justification
1	**F**	Open addressing stores every key **inside** the table, one per slot, so n ≤ m and **α ≤ 1**. (With chaining, α can be greater than 1 because chains can grow.)
2	**F**	A later key may have probed **past** that slot when it was inserted. If the slot becomes EMPTY, a search for that key stops early and wrongly reports "not found". Fix: mark the slot **DELETED** (a tombstone). Searches skip over it, and inserts may reuse it.
3	**F**	Perfect hashing is built for a **static** set. The second-level functions were chosen to avoid collisions **among the known keys only**. A new key can collide, so the bucket (or the whole table) must be rebuilt.
4	**T**	k mod 2^p keeps exactly the lowest p bits of k. Keys that differ only in their high bits all collide. That is why m is usually chosen to be a **prime not close to a power of 2**.
5	**T**	The expected chain length is α = 3. An unsuccessful search hashes in O(1), then scans the **whole** chain, so it costs Θ(1 + α) and examines about 3 elements.

PART B – QUESTION 2

a) Each key starts at k mod 10 and moves right until it finds an empty slot.

Key	h(k)	Slots tried	Final	Probes
42	2	2	2	1
12	2	2 ✗, 3	3	2
33	3	3 ✗, 4	4	2
52	2	2 ✗, 3 ✗, 4 ✗, 5	5	4
23	3	3 ✗, 4 ✗, 5 ✗, 6	6	4
7	7	7	7	1
17	7	7 ✗, 8	8	2

Slot	0	1	2	3	4	5	6	7	8	9
Key	–	–	42	12	33	52	23	7	17	–

I. Search 62: h = 2. Check slots 2, 3, 4, 5, 6, 7, 8 (all full and none equal 62), then slot 9 is **empty, so stop**. Result: **8 probes**.

II. Primary clustering. Occupied slots form one long block (slots 2 to 8). Any key that hashes **anywhere** in the block must walk to its end, and then it **extends** the block. So the bigger a cluster gets, the faster it grows (a slot right after a cluster of length L is filled with probability about (L+1)/m). Here, any search starting at slot 2 walks 7 full slots even though the table is only 70% full. **Fix:** quadratic probing or, better,

double hashing. Keys that start at the same slot then follow different step sizes, so they don't line up into one block.

b) Idea: use two passes with a hash map. Pass 1 counts each user's logins. Pass 2 walks the list **in its original order** and returns the first user whose count is 1. The second pass preserves the time order, which a single pass over the map would not.

```
FirstSingleLogin(Logins):
    H = empty hash map                    // user -> count
    for each u in Logins:                 // pass 1: count
        if u not in H: H[u] = 0
        H[u] = H[u] + 1
    for each u in Logins:                 // pass 2: in time order
        if H[u] == 1:
            return u
    return NONE
```

Counts: u7 = 2, u3 = 2, u9 = 1, u5 = 1. Pass 2 checks u7 (2), u3 (2), u7 (2), then **u9 (1), so it returns u9** ✓ (u5 also has count 1 but appears later).

Complexity: two passes of n steps, each O(1) expected, so **O(n) expected time** and **O(u) space** (u = number of distinct users ≤ n). Checking every user against the rest of the list would take $O(n^2)$.

PART B – QUESTION 3

a) Label the equal keys to track stability: A = [4_a, 1_a, 3, 4_b, 0, 1_b, 4_c]

Index	0	1	2	3	4
C after counting	1	2	0	1	3
C after prefix sums	1	3	3	4	7

Placements from the right (i = 7, 6, 5, …):

i	A[i]	C[A[i]]	Placed at	C[A[i]] after
7	4_c	7	B[7]	6
6	1_b	3	B[3]	2
5	0	1	B[1]	0
4	4_b	6	B[6]	5
3	3	4	B[4]	3
2	1_a	2	B[2]	1
1	4_a	5	B[5]	4

B position	1	2	3	4	5	6	7
After 3 placements	0	–	1_b	–	–	–	4_c
Final B	0	1_a	1_b	3	4_a	4_b	4_c

Forward loop (i = 1 to n): 4_a is placed first and takes the **last** 4-slot (B[7]), then 4_b takes B[6], then 4_c takes B[5]. The result is $[0, 1_b, 1_a, 3, 4_c, 4_b, 4_a]$.

The output is **still sorted** (the values are correct), but it is **NOT stable**: equal keys come out in **reverse** order. C[v] points at the **last** free slot for value v, so you must fill from the back of the input to keep equal keys in order.

Why it matters for radix sort: each pass must keep the order created by the earlier (less significant) passes among ties. An unstable pass destroys that order, so radix sort would give a wrong result.

b) With b-bit keys and r-bit digits: **d = ⌈b / r⌉ passes**, each costing $O(n + 2^r)$.

Digit choice	Buckets	# passes	Cost per pass	Total cost
Decimal digit	10	10 ($2^{32} \approx 4.29 \times 10^9$ has 10 digits)	n + 10	$\approx 1.0 \times 10^7$
1 byte (r = 8)	256	4	n + 256	$\approx 4.0 \times 10^6$
2 bytes (r = 16)	65,536	2	n + 65,536	**$\approx 2.13 \times 10^6$**

General formula: $T = (b / r) \cdot (n + 2^r)$.

Best: r = 16 (about 2.1×10^6). A comparison sort costs about $n \log_2 n \approx 10^6 \times 20 = 2 \times 10^7$, roughly **10× slower**.

Why not r = 32? That is 1 pass, but $2^{32} \approx 4.3 \times 10^9$ buckets, so the 2^r term dominates: huge time and memory for only $n = 10^6$ keys. Larger r means fewer passes but more buckets. The best choice is about **$r \approx \log_2 n$**, which keeps $2^r \approx n$ and gives $O(b \cdot n / \log n)$.

PART C – QUESTION 4

Why one path is enough: the closest key to K is either the **largest key ≤ K** (floor) or the **smallest key ≥ K** (ceiling). A normal BST search for K passes through both. Each time we go right, the current node is a candidate floor. Each time we go left, it is a candidate ceiling. Any node we skip lies on the wrong side of a node we already checked, so it is farther from K. So we only need to follow the search path for K and remember the best node seen.

```
Find-Closest(root, K):
    if root == NIL: return NIL
    best = root.key
    x = root
    while x != NIL:
        if |x.key - K| < |best - K|:
            best = x.key
        if x.key == K:
            return x.key                  // exact match, distance 0
        else if K < x.key:
            x = x.left
        else:
            x = x.right
    return best
```

Example (tree: 50 at the root; 30 and 70 as its children; 20, 40, 60, 80 as leaves):

K	Path (distance)	Answer
63	50 (13) → 70 (7) → 60 (3) → NIL	**60**
66	50 (16) → 70 (4) → 60 (6) → NIL	**70**
34	50 (16) → 30 (4) → 40 (6) → NIL	**30**

Complexity: one root-to-leaf path with O(1) work per node, so **O(h) = O(log n)** on a balanced BST and **O(1) space**. A full traversal comparing every tempo would take O(n).

PART C – QUESTION 5

a) In a preorder, the first key is the root. The keys after it that are **smaller** form the left subtree, and the rest form the right subtree. Apply this recursively:



b) Inorder: 10, 25, 30, 32, 40, 50, 60, 70 (sorted, as every BST inorder must be). **Postorder:** 10, 30, 32, 25, 50, 70, 60, 40

c) Idea: pass down allowed bounds (the same trick as BST validation). Read the preorder left to right with a **shared index**. Every node must fit in an interval (low, high) set by its ancestors. If the next key fits, it belongs here: create the node, then build its left subtree with bounds (low, key) and its right subtree with bounds (key, high). If it doesn't fit, it belongs to an ancestor, so return NIL without consuming it.

```
idx = 1                                          // shared position in pre[]

Build-From-Preorder(pre, n):
    idx = 1
    return Build(pre, n, -INFINITY, +INFINITY)

Build(pre, n, low, high):
    if idx > n: return NIL
    key = pre[idx]
    if key < low or key > high:
        return NIL                               // belongs to an ancestor
    x = new node(key)
    idx = idx + 1
    x.left  = Build(pre, n, low, key)            // smaller keys come next
    x.right = Build(pre, n, key, high)           // then the larger ones
    return x
```

Short trace: 40 fits (−∞, ∞), so it becomes the root. 25 fits (−∞, 40), so it goes left of 40. 10 fits (−∞, 25), so it goes left of 25. 32 does not fit (−∞, 10), so 10's left is NIL. 32 does not fit (10, 25), so 10's right is NIL. Back at 25: 32 fits (25, 40), so it goes right of 25. 30 fits (25, 32), so it goes left of 32. 60 does not fit, so it bubbles back up to the root and becomes the right child of 40 with bounds (40, ∞). This continues and reproduces the tree from (a) ✓

Complexity: each key is consumed (idx++) exactly **once**, creating one node. Every call either creates a node or returns NIL immediately, and each node makes 2 child calls, so there are at most 2n + 1 calls with O(1) work each. That gives **O(n) time** and **O(h) recursion space**.

Compared with inserting one by one: each insert costs O(depth), which is $O(n^2)$ total when the preorder is sorted (a skewed tree), or O(n log n) at best. The bounds method never searches from the root again.